

Assembly & Operation Instructions

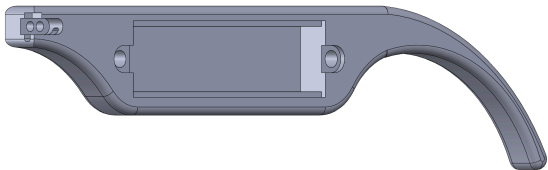
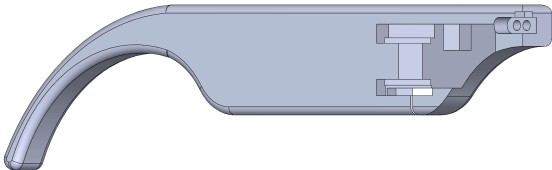
This document serves as a guide to build the smart glasses and get them working. All associated files, including CAD files for 3D printing, firmware for flashing, and PC side software are included in this repository. However, additional files such as older versions of CAD files and firmware used for debugging are not included.

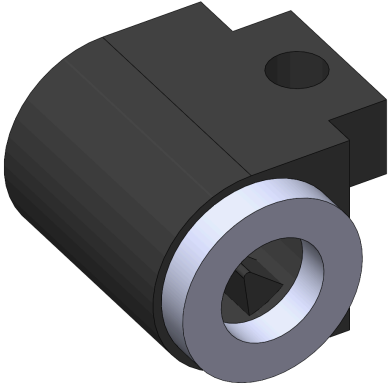
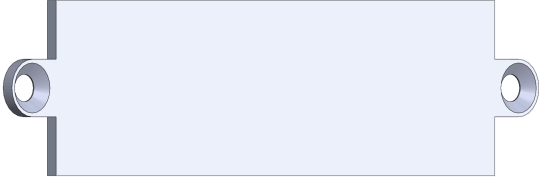
1. Assembling the Frames

1.1 3D Printing Mechanical Parts

The first step is to 3D print the various parts of the glasses, which are listed in the table below.

Table 1.1.1 – Parts to 3D Print

	Right Arm The Right Arm has two heat inserts for the battery door and two heat inserts for the right hinge. There is a channel through the plastic for the battery wires.
	Left Arm The Left Arm has two heat inserts for the left hinge and mating geometry for the Lens Holder Assembly.

	Lens Holder Assembly The Lens Holder Assembly consists of two parts: the Lens Holder and the Cap, which retains the lens. It also serves the purpose of locking the screen in the Left Arm. The geometry is set such that the focal distance of the lens is the distance between the screen and the lens. This assembly promotes a modular design, where different lenses can be used by simply swapping a new Lens Holder into the Right Arm.
	Frame The frame has four heat inserts for the hinges and a channel along the top for running the battery wires. Aside from the channel, the frame looks exactly like the frame of a standard pair of glasses.
	Battery Door The battery door is fastened into the Right Arm with two M2 screws, and fixes the battery.

1.2 Heated Inserts

The next step is to insert the heated inserts into the various 3D printed parts. We used M1 x 1.5mm L x 2mm OD heated inserts for securing the hinges. Four of these are inserted into the frame, and two are inserted into each of the arms, as mentioned in Table 1.1. The locations for the heated inserts should be quite self evident. M2 heated inserts were used to secure the battery door onto the right arm. These are inserted onto the inner face of the right arm, where the holes are. This should again be self-evident when looking at the right arm.

1.3 Securing the Lens

The next step in the assembly process is securing the convex lens in the lens holder. The lens used in our final version as of writing this document had a diameter of 8.8 mm and a focal length of 27 mm.

Place the lens into the lens holder such that it is resting on the axial locating feature, as shown in Figure 1.3.1

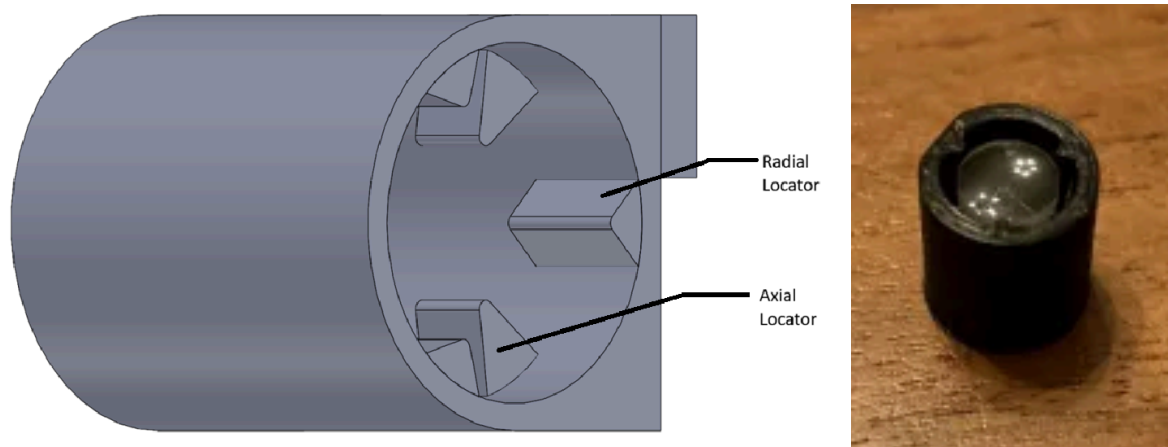
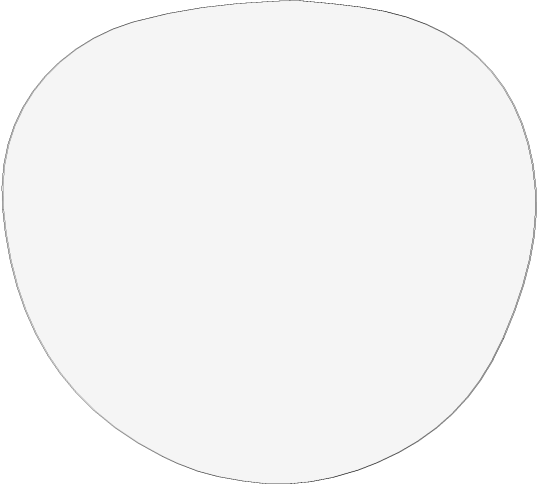


Figure 1.3.1 – Left: Lens holder design with radial and axial locating features. Right: 3D-printed proof-of-concept prototype. Lens cap not pictured.

Once that is done and the lens is sitting flat (this is very important) against the axial locator tabs, glue the cap to the lens holder using superglue, such that the lens is secured axially.

1.4 Laser Cutting Optics

Table 1.4.1 – Parts to laser cut

	Eye Lens The Eye Lenses are about 2.36mm thick and press fit into the frame. They are laser cut out of an acrylic sheet.
	Reflector The reflector is a slot with a 9mm radius and a 10mm centre-to-centre distance. It is laser cut out of a thin reflective sheet and allows the reflection from the screen to be observed as well as the surroundings behind it, giving a projecting effect.

2. Electrical Assembly

2.1 PCB Assembly

If you do not already have the PCB, you can access the files in the [PCB repository](#). We ordered ours from JLCPCB.

2.2 Soldering Wires to the FPC/ZIF Connector

This is the most tedious part of the assembly process. These smart glasses used a 0.32 inch micro-OLED display, specifically the one shown in Figure 2.2.1.

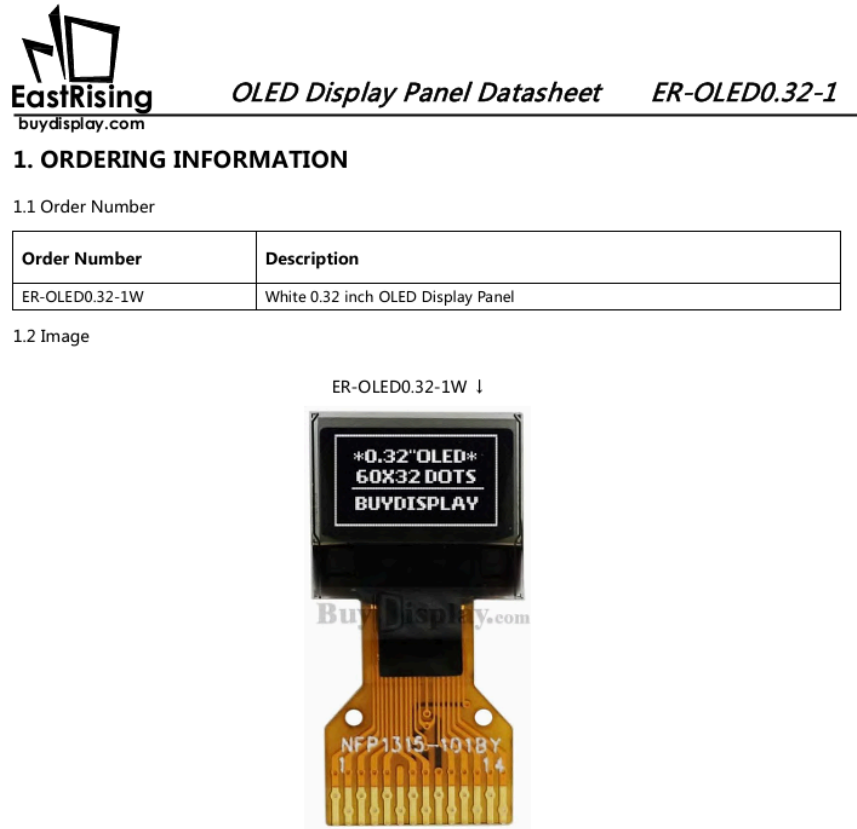


Figure 2.2.1 – OLED Display used in these Smart Glasses

These micro-OLED displays shipped with ZIF/FPC connectors. The pitch of the pads on the display are non-standard and it is not easy to find alternatives. We tried soldering wires directly to the flex PCB of the display, but were unsuccessful. The best option we found was to solder wires (30 gauge wire wrapping wire in our case) to each pin of the FPC connector, as shown in Figure 2.2.2.

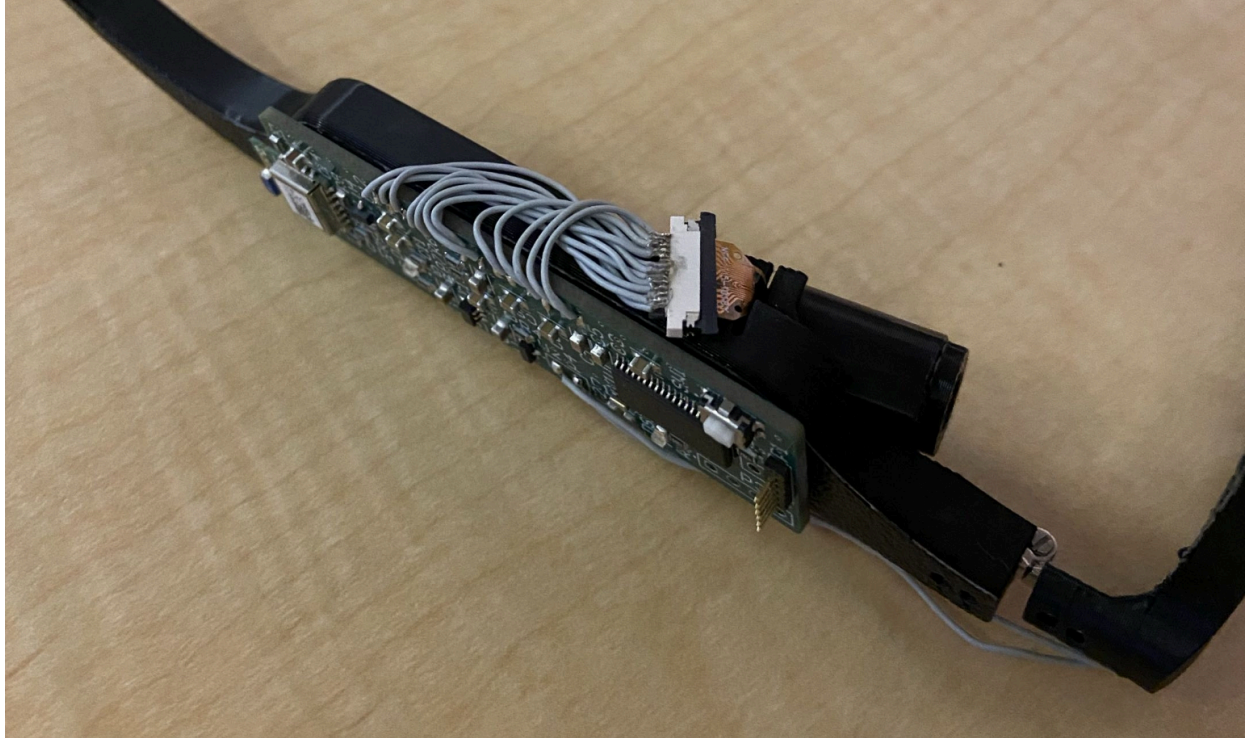


Figure 2.2.2 – Wires soldered to FPC connector

Note that Pin 6 does not have any wire soldered to it, as this pin is NC as indicated in the display's datasheet.

Once this has been done, solder each wire to its corresponding pad on the PCB, as shown in Figure 2.2.3.

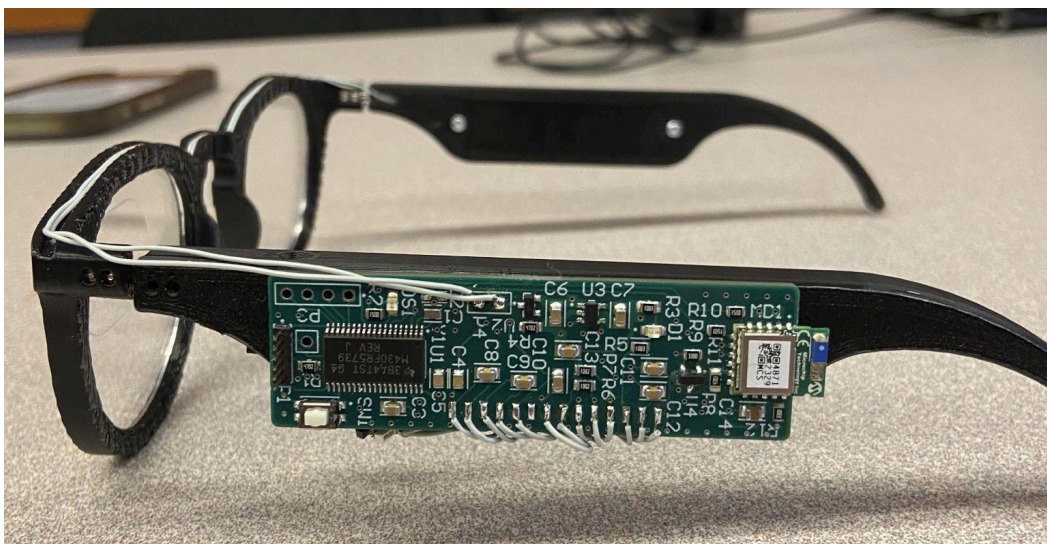


Figure 2.2.3 – Wires soldered to their corresponding pins on the PCB

3. Flashing Firmware to the PCB

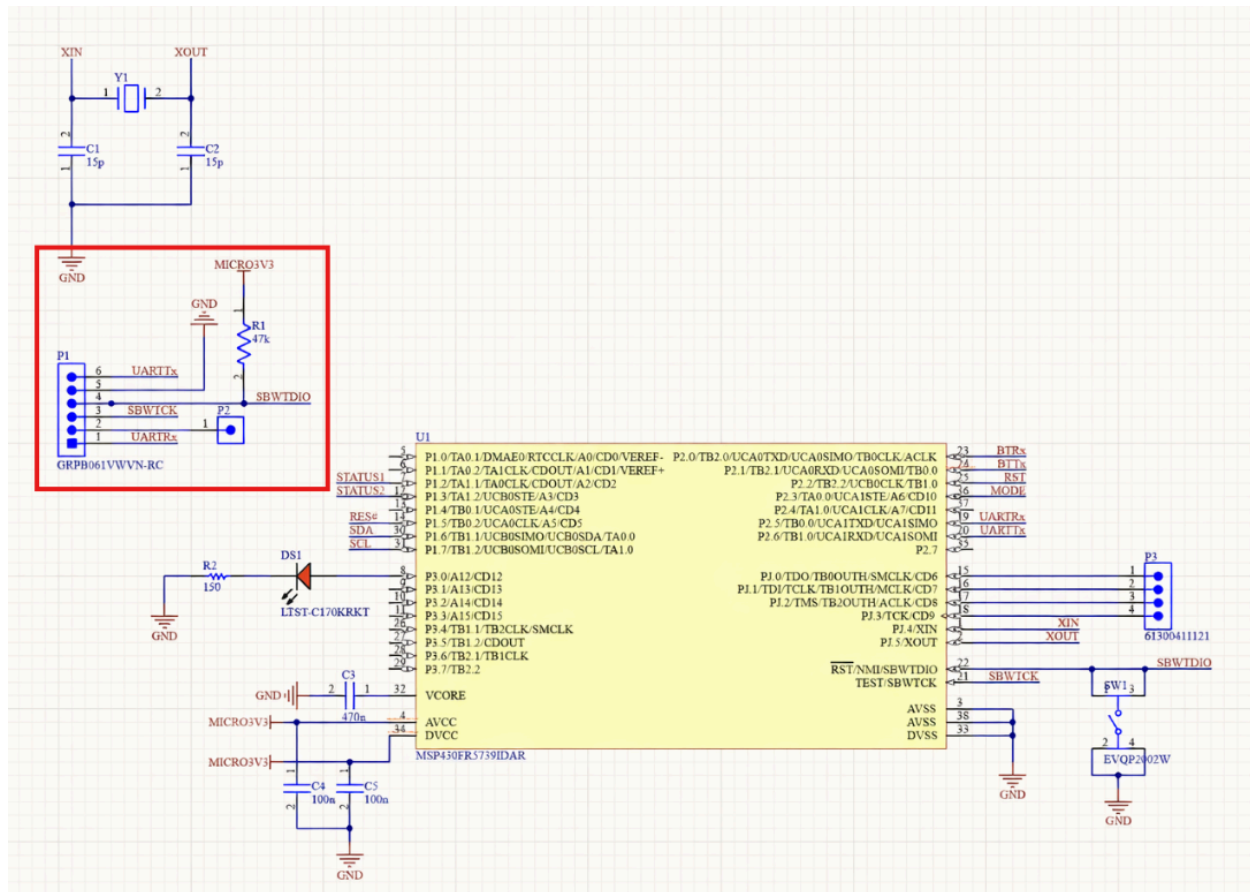


Figure 3.1 – MCU page from schematic with Spy-Bi-Wire interface boxed

Flashing is done over the Spy-Bi-Wire interface, which is shown in the schematic page in Fig. 3.1. For flashing the PCBs, the TI product that you need is [here](#). That comes with the USB connector to connect the TI Board to your computer, and you also need a 6-pin 0.05" (1.27mm) pitch connector (female-female) to go from the TI Board to our PCBs. The development environment for TI is [CCSTUDIO](#).

The connector is at the MCU end of the PCB, and can be seen at the left side of the PCB in Fig. 2.2.3.

4. Setting Up Docker Desktop for Valhalla

Download [Docker Desktop](#) and run either of the following commands in Windows Command Prompt (CMD):

If you have already downloaded the map tiles separately (from <https://download.geofabrik.de/>):

```
docker run -dt ^
  --name valhalla_bc ^
  -p 8002:8002 ^
  -v "C:\Path\To\Your\custom_files:/custom_files" ^
  -e tile_urls=/custom_files/british-columbia-latest.osm.pbf ^
  -e build_tar=True ^
  -e serve_tiles=True ^
  -e update_existing_config=True ^
  ghcr.io/nilsnolde/docker-valhalla/valhalla:latest
```

To do everything in one step:

```
docker run -dt ^
  --name valhalla_bc ^
  -p 8002:8002 ^
  -v "C:\Path\To\Your\custom_files:/custom_files" ^
  -e
tile_urls=https://download.geofabrik.de/north-america/canada/british-columbia-latest.o
sm.pbf ^
  -e build_tar=True ^
  -e serve_tiles=True ^
  -e update_existing_config=True ^
  ghcr.io/nilsnolde/docker-valhalla/valhalla:latest
```

This environment is used for generating the navigation instructions.